Task 1: Hospital Management Database

Scenario: A hospital needs a system to manage its doctors, patients, and appointments.

Requirements:

1. **Schema Design:** Create a normalized schema for `Doctors`, `Patients`, and `Appointments` tables.
   - Include primary keys, appropriate data types, NOT NULL constraints, unique constraints (e.g., doctor phone/email, patient phone/email), and foreign key relationships with `ON DELETE CASCADE` where applicable.
   - Ensure an appointment is uniquely identified by a doctor and a date/time (prevent double-booking).
2. **Sample Data:** Insert sample data into all tables.
3. **SQL Queries:** Write SQL queries to:
   - Retrieve all appointments for a specific doctor (e.g., Doctor ID 1).
   - Retrieve the full appointment history for a specific patient (e.g., Patient ID 2).
   - Count the total number of unique patients treated by each doctor.

```python
import sqlite3

def setup_hospital_database():
    conn = sqlite3.connect(':memory:') # Use in-memory database for demonstration
    cursor = conn.cursor()

    # Schema Creation
    cursor.execute('''
        CREATE TABLE Doctors (
            doctor_id INTEGER PRIMARY KEY AUTOINCREMENT,
            first_name TEXT NOT NULL,
            last_name TEXT NOT NULL,
            specialization TEXT NOT NULL,
            phone_number TEXT UNIQUE NOT NULL,
            email TEXT UNIQUE NOT NULL
        );
    ''')

    cursor.execute('''
        CREATE TABLE Patients (
            patient_id INTEGER PRIMARY KEY AUTOINCREMENT,
            first_name TEXT NOT NULL,
            last_name TEXT NOT NULL,
            date_of_birth DATE NOT NULL,
            gender TEXT,
            address TEXT,
            phone_number TEXT UNIQUE NOT NULL,
            email TEXT UNIQUE
        );
```

```
    ''')

    cursor.execute('''
        CREATE TABLE Appointments (
            appointment_id INTEGER PRIMARY KEY AUTOINCREMENT,
            patient_id INTEGER NOT NULL,
            doctor_id INTEGER NOT NULL,
            appointment_date DATETIME NOT NULL,
            reason TEXT,
            FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
ON DELETE CASCADE,
            FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id) ON
DELETE CASCADE,
            UNIQUE (doctor_id, appointment_date)
        );
    ''')

    print("Hospital Management database schema created successfully.")

    # Sample Data Insertion
    cursor.execute("INSERT INTO Doctors (first_name, last_name,
specialization, phone_number, email) VALUES ('Alice', 'Smith',
'Cardiology', '555-1001', 'alice.smith@hospital.com')")
    cursor.execute("INSERT INTO Doctors (first_name, last_name,
specialization, phone_number, email) VALUES ('Bob', 'Johnson',
'Pediatrics', '555-1002', 'bob.j@hospital.com')")
    cursor.execute("INSERT INTO Doctors (first_name, last_name,
specialization, phone_number, email) VALUES ('Charlie', 'Brown',
'Dermatology', '555-1003', 'charlie.b@hospital.com')")

    cursor.execute("INSERT INTO Patients (first_name, last_name,
date_of_birth, gender, address, phone_number, email) VALUES ('Eva',
'Green', '1990-05-15', 'Female', '123 Main St', '555-2001',
'eva.g@email.com')")
    cursor.execute("INSERT INTO Patients (first_name, last_name,
date_of_birth, gender, address, phone_number, email) VALUES ('David',
'Lee', '1985-11-22', 'Male', '456 Oak Ave', '555-2002',
'david.l@email.com')")
    cursor.execute("INSERT INTO Patients (first_name, last_name,
date_of_birth, gender, address, phone_number, email) VALUES ('Grace',
'Chan', '2000-01-01', 'Female', '789 Pine Ln', '555-2003',
'grace.c@email.com')")

    cursor.execute("INSERT INTO Appointments (patient_id, doctor_id,
appointment_date, reason) VALUES (1, 1, '2023-10-26 10:00:00',
'Routine Checkup')")
    cursor.execute("INSERT INTO Appointments (patient_id, doctor_id,
appointment_date, reason) VALUES (2, 1, '2023-10-26 11:00:00',
'Follow-up')")
    cursor.execute("INSERT INTO Appointments (patient_id, doctor_id,
```

```
appointment_date, reason) VALUES (1, 2, '2023-10-27 09:30:00',
'Vaccination')")
    cursor.execute("INSERT INTO Appointments (patient_id, doctor_id,
appointment_date, reason) VALUES (3, 2, '2023-10-27 10:30:00', 'Annual
Physical')")
    cursor.execute("INSERT INTO Appointments (patient_id, doctor_id,
appointment_date, reason) VALUES (2, 3, '2023-10-28 14:00:00', 'Skin
Rash')")

    conn.commit()

    print("Sample data inserted.")

    # Query 1: List all appointments for a specific doctor (e.g.,
Doctor ID 1)
    print("\n--- Query 1: Retrieve all appointments for Doctor ID 1
---")
    doctor_id_to_query = 1
    cursor.execute('''
        SELECT
            A.appointment_id,
            A.appointment_date,
            A.reason,
            P.first_name AS patient_first_name,
            P.last_name AS patient_last_name
        FROM
            Appointments A
        JOIN
            Patients P ON A.patient_id = P.patient_id
        WHERE
            A.doctor_id = ?
        ORDER BY A.appointment_date;
    ''', (doctor_id_to_query,))
    appointments_for_doctor = cursor.fetchall()
    for appt in appointments_for_doctor:
        print(appt)

    # Query 2: Retrieve patient history by patient ID (e.g., Patient
ID 2)
    print("\n--- Query 2: Retrieve patient history for Patient ID 2
---")
    patient_id_to_query = 2
    cursor.execute('''
        SELECT
            A.appointment_id,
            A.appointment_date,
            A.reason,
            D.first_name AS doctor_first_name,
            D.last_name AS doctor_last_name,
            D.specialization
```

```
        FROM
            Appointments A
        JOIN
            Doctors D ON A.doctor_id = D.doctor_id
        WHERE
            A.patient_id = ?
        ORDER BY A.appointment_date;
    ''', (patient_id_to_query,))
    patient_history = cursor.fetchall()
    for hist in patient_history:
        print(hist)

    # Query 3: Count total patients treated by each doctor
    print("\n--- Query 3: Count total patients treated by each doctor
---")
    cursor.execute('''
        SELECT
            D.doctor_id,
            D.first_name,
            D.last_name,
            D.specialization,
            COUNT(DISTINCT A.patient_id) AS total_patients_treated
        FROM
            Doctors D
        LEFT JOIN
            Appointments A ON D.doctor_id = A.doctor_id
        GROUP BY
            D.doctor_id, D.first_name, D.last_name, D.specialization
        ORDER BY
            total_patients_treated DESC, D.last_name;
    ''')
    patients_by_doctor = cursor.fetchall()
    for doc_count in patients_by_doctor:
        print(doc_count)

    conn.close()

setup_hospital_database()
```

```
Hospital Management database schema created successfully.
Sample data inserted.

--- Query 1: Retrieve all appointments for Doctor ID 1 ---
(1, '2023-10-26 10:00:00', 'Routine Checkup', 'Eva', 'Green')
(2, '2023-10-26 11:00:00', 'Follow-up', 'David', 'Lee')

--- Query 2: Retrieve patient history for Patient ID 2 ---
(2, '2023-10-26 11:00:00', 'Follow-up', 'Alice', 'Smith',
'Cardiology')
(5, '2023-10-28 14:00:00', 'Skin Rash', 'Charlie', 'Brown',
```

```
'Dermatology')

--- Query 3: Count total patients treated by each doctor ---
(2, 'Bob', 'Johnson', 'Pediatrics', 2)
(1, 'Alice', 'Smith', 'Cardiology', 2)
(3, 'Charlie', 'Brown', 'Dermatology', 1)
```

Task 2: Online Shopping Database

Scenario: Design a simplified database for an e-commerce platform.

Requirements:

1. **Schema Design:** Create tables for `Users`, `Products`, `Orders`, and `OrderDetails`.
   - Include primary keys, data types, NOT NULL, UNIQUE constraints (e.g., username, email), and CHECK constraints (e.g., stock quantity >= 0, order quantity > 0).
   - Define foreign key relationships with `ON DELETE CASCADE`.
   - Consider storing `price_at_purchase` in `OrderDetails` to maintain historical accuracy for past orders.
2. **Sample Data:** Populate the tables with example data.
3. **SQL Queries:** Write SQL queries to:
   - Retrieve all orders placed by a specific user (e.g., User ID 1), including the products in each order.
   - Find the most purchased product across all orders.
   - Calculate the total revenue (from completed orders) for a specific month (e.g., January 2023).

```python
import sqlite3

def setup_ecommerce_database():
    conn = sqlite3.connect(':memory:') # Use in-memory database for
demonstration
    cursor = conn.cursor()

    # Schema Creation
    cursor.execute('''
        CREATE TABLE Users (
            user_id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT UNIQUE NOT NULL,
            email TEXT UNIQUE NOT NULL,
            password_hash TEXT NOT NULL,
            registration_date DATETIME DEFAULT CURRENT_TIMESTAMP
        );
    ''')

    cursor.execute('''
        CREATE TABLE Products (
            product_id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
```

```
            description TEXT,
            price DECIMAL(10, 2) NOT NULL,
            stock_quantity INTEGER NOT NULL CHECK (stock_quantity >=
0)
        );
    ''')

    cursor.execute('''
        CREATE TABLE Orders (
            order_id INTEGER PRIMARY KEY AUTOINCREMENT,
            user_id INTEGER NOT NULL,
            order_date DATETIME DEFAULT CURRENT_TIMESTAMP,
            total_amount DECIMAL(10, 2) NOT NULL,
            status TEXT NOT NULL DEFAULT 'Pending',
            FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE
CASCADE
        );
    ''')

    cursor.execute('''
        CREATE TABLE OrderDetails (
            order_detail_id INTEGER PRIMARY KEY AUTOINCREMENT,
            order_id INTEGER NOT NULL,
            product_id INTEGER NOT NULL,
            quantity INTEGER NOT NULL CHECK (quantity > 0),
            price_at_purchase DECIMAL(10, 2) NOT NULL, -- Store price
at time of purchase
            FOREIGN KEY (order_id) REFERENCES Orders(order_id) ON
DELETE CASCADE,
            FOREIGN KEY (product_id) REFERENCES Products(product_id)
ON DELETE CASCADE
        );
    ''')

    print("E-commerce database schema created successfully.")

    # Sample Data Insertion
    cursor.execute("INSERT INTO Users (username, email, password_hash)
VALUES ('johndoe', 'john.doe@example.com', 'hashedpassword1')")
    cursor.execute("INSERT INTO Users (username, email, password_hash)
VALUES ('janesmith', 'jane.smith@example.com', 'hashedpassword2')")
    cursor.execute("INSERT INTO Users (username, email, password_hash)
VALUES ('petert', 'peter.t@example.com', 'hashedpassword3')")

    cursor.execute("INSERT INTO Products (name, description, price,
stock_quantity) VALUES ('Laptop', 'High performance laptop', 1200.00,
50)")
    cursor.execute("INSERT INTO Products (name, description, price,
stock_quantity) VALUES ('Mouse', 'Wireless ergonomic mouse', 25.00,
200)")
```

```
    cursor.execute("INSERT INTO Products (name, description, price,
stock_quantity) VALUES ('Keyboard', 'Mechanical gaming keyboard',
75.00, 100)")
    cursor.execute("INSERT INTO Products (name, description, price,
stock_quantity) VALUES ('Monitor', '27-inch 4K monitor', 300.00, 75)")

    cursor.execute("INSERT INTO Orders (user_id, total_amount, status,
order_date) VALUES (1, 1225.00, 'Completed', '2023-01-15 10:00:00')")
    cursor.execute("INSERT INTO Orders (user_id, total_amount, status,
order_date) VALUES (1, 375.00, 'Processing', '2023-02-20 14:30:00')")
    cursor.execute("INSERT INTO Orders (user_id, total_amount, status,
order_date) VALUES (2, 25.00, 'Completed', '2023-01-20 11:00:00')")
    cursor.execute("INSERT INTO Orders (user_id, total_amount, status,
order_date) VALUES (3, 1275.00, 'Shipped', '2023-02-25 09:00:00')")
    cursor.execute("INSERT INTO Orders (user_id, total_amount, status,
order_date) VALUES (1, 300.00, 'Pending', '2023-03-01 16:00:00')")

    cursor.execute("INSERT INTO OrderDetails (order_id, product_id,
quantity, price_at_purchase) VALUES (1, 1, 1, 1200.00)")
    cursor.execute("INSERT INTO OrderDetails (order_id, product_id,
quantity, price_at_purchase) VALUES (1, 2, 1, 25.00)")
    cursor.execute("INSERT INTO OrderDetails (order_id, product_id,
quantity, price_at_purchase) VALUES (2, 3, 5, 75.00)")
    cursor.execute("INSERT INTO OrderDetails (order_id, product_id,
quantity, price_at_purchase) VALUES (3, 2, 1, 25.00)")
    cursor.execute("INSERT INTO OrderDetails (order_id, product_id,
quantity, price_at_purchase) VALUES (4, 1, 1, 1200.00)")
    cursor.execute("INSERT INTO OrderDetails (order_id, product_id,
quantity, price_at_purchase) VALUES (4, 3, 1, 75.00)")
    cursor.execute("INSERT INTO OrderDetails (order_id, product_id,
quantity, price_at_purchase) VALUES (5, 4, 1, 300.00)")

    conn.commit()

    print("Sample data inserted.")

    # Query 1: Retrieve all orders by a specific user (e.g., user_id
1)
    print("\n--- Query 1: Retrieve all orders by a specific user (User
ID 1) ---")
    user_id_to_query = 1
    cursor.execute('''
        SELECT
            O.order_id,
            O.order_date,
            O.total_amount,
            O.status,
            GROUP_CONCAT(P.name || ' (x' || OD.quantity || ')', '; ')
AS products_ordered
        FROM
```

```
            Orders O
        JOIN
            OrderDetails OD ON O.order_id = OD.order_id
        JOIN
            Products P ON OD.product_id = P.product_id
        WHERE
            O.user_id = ?
        GROUP BY
            O.order_id, O.order_date, O.total_amount, O.status
        ORDER BY
            O.order_date DESC;
    ''', (user_id_to_query,))
    user_orders = cursor.fetchall()
    for order in user_orders:
        print(order)

    # Query 2: Find the most purchased product
    print("\n--- Query 2: Find the most purchased product ---")
    cursor.execute('''
        SELECT
            P.product_id,
            P.name,
            SUM(OD.quantity) AS total_quantity_purchased
        FROM
            OrderDetails OD
        JOIN
            Products P ON OD.product_id = P.product_id
        GROUP BY
            P.product_id, P.name
        ORDER BY
            total_quantity_purchased DESC
        LIMIT 1;
    ''')
    most_purchased_product = cursor.fetchone()
    print(most_purchased_product)

    # Query 3: Calculate total revenue in a given month (e.g., January
2023)
    print("\n--- Query 3: Calculate total revenue in January 2023
(Completed Orders Only) ---")
    month_to_query = '2023-01'
    cursor.execute('''
        SELECT
            SUM(OD.quantity * OD.price_at_purchase) AS total_revenue
        FROM
            Orders O
        JOIN
            OrderDetails OD ON O.order_id = OD.order_id
        WHERE
```

```
            STRFTIME('%Y-%m', O.order_date) = ?
            AND O.status = 'Completed';
    ''', (month_to_query,))
    total_revenue = cursor.fetchone()
    print(total_revenue)

    conn.close()

setup_ecommerce_database()
```

```
E-commerce database schema created successfully.
Sample data inserted.

--- Query 1: Retrieve all orders by a specific user (User ID 1) ---
(5, '2023-03-01 16:00:00', 300, 'Pending', 'Monitor (x1)')
(2, '2023-02-20 14:30:00', 375, 'Processing', 'Keyboard (x5)')
(1, '2023-01-15 10:00:00', 1225, 'Completed', 'Laptop (x1); Mouse
(x1)')

--- Query 2: Find the most purchased product ---
(3, 'Keyboard', 6)

--- Query 3: Calculate total revenue in January 2023 (Completed Orders
Only) ---
(1250,)
```

Task 3: Library Management Database

Scenario: A library wants to manage its book collection, members, and loan transactions.

Requirements:

1. **Schema Design:** Create normalized tables for `Books`, `Members`, and `Loans`.
   - Include primary keys, data types, NOT NULL constraints, unique constraints (e.g., ISBN for books, email for members).
   - Add a `CHECK` constraint for `available_copies` (must be >= 0).
   - Define foreign key relationships with `ON DELETE CASCADE`.
   - Ensure a unique constraint for `Loans` to prevent a member from borrowing the exact same book on the exact same date multiple times.
2. **Suggested Indexing Strategy:** Outline an indexing strategy for improved query performance, especially for frequently accessed columns and foreign keys.
3. **Sample Data:** Insert realistic sample data, including some unreturned and overdue loans.
4. **SQL Queries:** Write SQL queries to:
   - List all books that are currently issued (not yet returned).
   - Identify all overdue books (books whose `loan_date` was more than 30 days ago and have not been returned).
   - Count the total number of books loaned by each member (including members who haven't loaned any books).

```python
import sqlite3
from datetime import datetime, timedelta

def setup_library_database():
    conn = sqlite3.connect(':memory:')  # Use in-memory database for
demonstration
    cursor = conn.cursor()

    # Schema Creation
    cursor.execute('''
        CREATE TABLE Books (
            book_id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT NOT NULL,
            author TEXT NOT NULL,
            isbn TEXT UNIQUE NOT NULL,
            publication_year INTEGER,
            available_copies INTEGER NOT NULL CHECK (available_copies
>= 0)
        );
    ''')

    cursor.execute('''
        CREATE TABLE Members (
            member_id INTEGER PRIMARY KEY AUTOINCREMENT,
            first_name TEXT NOT NULL,
            last_name TEXT NOT NULL,
            email TEXT UNIQUE NOT NULL,
            phone_number TEXT UNIQUE
        );
    ''')

    cursor.execute('''
        CREATE TABLE Loans (
            loan_id INTEGER PRIMARY KEY AUTOINCREMENT,
            book_id INTEGER NOT NULL,
            member_id INTEGER NOT NULL,
            loan_date DATE NOT NULL,
            return_date DATE, -- NULL if not yet returned
            FOREIGN KEY (book_id) REFERENCES Books(book_id) ON DELETE
CASCADE,
            FOREIGN KEY (member_id) REFERENCES Members(member_id) ON
DELETE CASCADE,
            UNIQUE (book_id, member_id, loan_date) -- Prevent
duplicate loans for the same book/member on the same day
        );
    ''')

    print("Library Management database schema created successfully.")

    # Suggested Indexing Strategy for Faster Queries:
```

```
    # 1. Foreign Keys are crucial for JOIN performance:
    #    CREATE INDEX idx_loans_book_id ON Loans(book_id);
    #    CREATE INDEX idx_loans_member_id ON Loans(member_id);
    # 2. Frequently queried columns:
    #    CREATE INDEX idx_books_title ON Books(title);
    #    CREATE INDEX idx_books_author ON Books(author);
    #    CREATE INDEX idx_books_isbn ON Books(isbn);
    #    CREATE INDEX idx_members_email ON Members(email);
    #    CREATE INDEX idx_loans_loan_date ON Loans(loan_date); -- For
date-based queries (e.g., overdue books)
    # 3. For 'currently issued' books, an index on return_date for
NULL values can be beneficial:
    #    CREATE INDEX idx_loans_return_date ON Loans(return_date);

    # Sample Data Insertion
    cursor.execute("INSERT INTO Books (title, author, isbn,
publication_year, available_copies) VALUES ('The Great Gatsby', 'F.
Scott Fitzgerald', '978-0743273565', 1925, 3)")
    cursor.execute("INSERT INTO Books (title, author, isbn,
publication_year, available_copies) VALUES ('1984', 'George Orwell',
'978-0451524935', 1949, 2)")
    cursor.execute("INSERT INTO Books (title, author, isbn,
publication_year, available_copies) VALUES ('To Kill a Mockingbird',
'Harper Lee', '978-0061120084', 1960, 1)")
    cursor.execute("INSERT INTO Books (title, author, isbn,
publication_year, available_copies) VALUES ('Pride and Prejudice',
'Jane Austen', '978-0141439518', 1813, 0)") # No copies available

    cursor.execute("INSERT INTO Members (first_name, last_name, email,
phone_number) VALUES ('John', 'Doe', 'john.doe@example.com', '111-222-
3333')")
    cursor.execute("INSERT INTO Members (first_name, last_name, email,
phone_number) VALUES ('Jane', 'Smith', 'jane.smith@example.com', '444-
555-6666')")
    cursor.execute("INSERT INTO Members (first_name, last_name, email,
phone_number) VALUES ('Alice', 'Johnson', 'alice.j@example.com',
NULL)")

    # Current date for loans
    today = datetime.now().strftime('%Y-%m-%d')
    thirty_one_days_ago = (datetime.now() -
timedelta(days=31)).strftime('%Y-%m-%d')

    cursor.execute(f"INSERT INTO Loans (book_id, member_id, loan_date,
return_date) VALUES (1, 1, '{today}', NULL)") # John borrows Gatsby
    cursor.execute(f"INSERT INTO Loans (book_id, member_id, loan_date,
return_date) VALUES (2, 2, '{today}', NULL)") # Jane borrows 1984
    cursor.execute(f"INSERT INTO Loans (book_id, member_id, loan_date,
return_date) VALUES (3, 1, '{today}', NULL)") # John borrows
Mockingbird
```

```
    cursor.execute(f"INSERT INTO Loans (book_id, member_id, loan_date, return_date) VALUES (1, 2, '2023-09-01', '2023-09-15')") # Jane borrowed Gatsby and returned
    cursor.execute(f"INSERT INTO Loans (book_id, member_id, loan_date, return_date) VALUES (2, 1, '{thirty_one_days_ago}', NULL)") # John borrowed 1984, now overdue

    conn.commit()

    print("Sample data inserted.")

    # Query 1: Retrieve all books currently issued
    print("\n--- Query 1: All books currently issued ---")
    cursor.execute('''
        SELECT
            B.title, B.author, B.isbn,
            M.first_name AS member_first_name,
            M.last_name AS member_last_name,
            L.loan_date
        FROM
            Loans L
        JOIN
            Books B ON L.book_id = B.book_id
        JOIN
            Members M ON L.member_id = M.member_id
        WHERE
            L.return_date IS NULL;
    ''')
    currently_issued_books = cursor.fetchall()
    for book in currently_issued_books:
        print(book)

    # Query 2: Find overdue books (loan date > 30 days)
    print("\n--- Query 2: Overdue books ---")
    cursor.execute(f'''
        SELECT
            B.title, B.author,
            M.first_name AS member_first_name,
            M.last_name AS member_last_name,
            L.loan_date
        FROM
            Loans L
        JOIN
            Books B ON L.book_id = B.book_id
        JOIN
            Members M ON L.member_id = M.member_id
        WHERE
            L.return_date IS NULL AND L.loan_date < DATE('now', '-30 days');
    ''')
```

```
    overdue_books = cursor.fetchall()
    for book in overdue_books:
        print(book)

    # Query 3: Count number of books loaned by each member
    print("\n--- Query 3: Books loaned by each member ---")
    cursor.execute('''
        SELECT
            M.first_name, M.last_name,
            COUNT(L.loan_id) AS total_books_loaned
        FROM
            Members M
        LEFT JOIN
            Loans L ON M.member_id = L.member_id
        GROUP BY
            M.member_id, M.first_name, M.last_name
        ORDER BY
            total_books_loaned DESC, M.last_name;
    ''')
    books_loaned_by_member = cursor.fetchall()
    for member_loan_count in books_loaned_by_member:
        print(member_loan_count)

    conn.close()

setup_library_database()
```

```
Library Management database schema created successfully.
Sample data inserted.

--- Query 1: All books currently issued ---
('The Great Gatsby', 'F. Scott Fitzgerald', '978-0743273565', 'John',
'Doe', '2026-03-27')
('1984', 'George Orwell', '978-0451524935', 'Jane', 'Smith', '2026-03-
27')
('To Kill a Mockingbird', 'Harper Lee', '978-0061120084', 'John',
'Doe', '2026-03-27')
('1984', 'George Orwell', '978-0451524935', 'John', 'Doe', '2026-02-
24')

--- Query 2: Overdue books ---
('1984', 'George Orwell', 'John', 'Doe', '2026-02-24')

--- Query 3: Books loaned by each member ---
('John', 'Doe', 3)
('Jane', 'Smith', 2)
('Alice', 'Johnson', 0)
```

Task 4: Query Optimization Demonstration

Scenario: Understanding and demonstrating the performance benefits of query optimization techniques in SQL.

Requirements:

1. **Database Setup:** Create a simple database with two tables, `Authors` and `Books`, with a foreign key relationship.
2. **Sample Data:** Populate the tables with enough data to demonstrate a query.
3. **Original Query:** Write a query that fetches books by authors from a specific country using a subquery with an `IN` clause (e.g., `WHERE author_id IN (SELECT author_id FROM Authors WHERE country='UK')`).
4. **Optimized Query:** Rewrite the same query using a `JOIN` operation (e.g., `SELECT ... FROM Books JOIN Authors ON ... WHERE Authors.country='UK'`).
5. **Performance Comparison:** Measure the execution time for both the original and optimized queries.
6. **Verification:** Confirm that both queries return the same results.
7. **Explanation:** Provide a clear explanation of why the `JOIN` approach is generally more efficient than the subquery with `IN` for larger datasets, highlighting the role of indexes.

```
import sqlite3
import time

def demonstrate_query_optimization():
    conn = sqlite3.connect(':memory:')
    cursor = conn.cursor()

    # 1. Create Tables
    cursor.execute('''
        CREATE TABLE Authors (
            author_id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            country TEXT NOT NULL
        );
    ''')

    cursor.execute('''
        CREATE TABLE Books (
            book_id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT NOT NULL,
            author_id INTEGER NOT NULL,
            FOREIGN KEY (author_id) REFERENCES Authors(author_id) ON
DELETE CASCADE
        );
    ''')
    print("Tables 'Authors' and 'Books' created successfully.")

    # 2. Insert Sample Data
    cursor.execute("INSERT INTO Authors (name, country) VALUES ('Jane
```

```
Austen', 'UK')")
    cursor.execute("INSERT INTO Authors (name, country) VALUES
('Charles Dickens', 'UK')")
    cursor.execute("INSERT INTO Authors (name, country) VALUES ('Mark
Twain', 'USA')")
    cursor.execute("INSERT INTO Authors (name, country) VALUES ('Leo
Tolstoy', 'Russia')")

    cursor.execute("INSERT INTO Books (title, author_id) VALUES
('Pride and Prejudice', 1)")
    cursor.execute("INSERT INTO Books (title, author_id) VALUES
('Sense and Sensibility', 1)")
    cursor.execute("INSERT INTO Books (title, author_id) VALUES
('Great Expectations', 2)")
    cursor.execute("INSERT INTO Books (title, author_id) VALUES
('Oliver Twist', 2)")
    cursor.execute("INSERT INTO Books (title, author_id) VALUES ('The
Adventures of Tom Sawyer', 3)")
    cursor.execute("INSERT INTO Books (title, author_id) VALUES ('War
and Peace', 4)")
    conn.commit()
    print("Sample data inserted.")

    # Optional: Add index to author_id in Books for optimized query
performance
    # In a real scenario, this index would exist for foreign keys
    cursor.execute('CREATE INDEX idx_books_author_id ON
Books(author_id);')
    print("Index on Books.author_id created.")

    print("\n--- Querying for books by UK authors ---")

    # 3. Original Query (using subquery)
    original_query = """
        SELECT * FROM Books
        WHERE author_id IN (
            SELECT author_id FROM Authors WHERE country='UK'
        );
    """

    start_time = time.time()
    cursor.execute(original_query)
    original_results = cursor.fetchall()
    original_time = time.time() - start_time

    print("\nOriginal Query Results (Books by UK authors):")
    for book in original_results:
        print(book)
    print(f"Original Query Time: {original_time:.6f} seconds")
```

```
    # 4. Optimized Query (using JOIN)
    optimized_query = """
        SELECT B.* FROM Books B
        JOIN Authors A ON B.author_id = A.author_id
        WHERE A.country = 'UK';
    """

    start_time = time.time()
    cursor.execute(optimized_query)
    optimized_results = cursor.fetchall()
    optimized_time = time.time() - start_time

    print("\nOptimized Query Results (Books by UK authors):")
    for book in optimized_results:
        print(book)
    print(f"Optimized Query Time: {optimized_time:.6f} seconds")

    # 5. Verify Correctness
    print("\nVerification: Results are identical - ", original_results
== optimized_results)

    conn.close()

    print("\n--- Explanation of Optimization ---")
    print("Both queries return the same correct results: all books
written by authors from the UK.")
    print("For larger datasets, the JOINed query (optimized version)
is generally faster than the subquery version. Here's why:")
    print("1.  **Subquery (IN clause):** The inner subquery `(SELECT
author_id FROM Authors WHERE country='UK')` is executed first to get a
list of author_ids. Then, the outer query `SELECT * FROM Books WHERE
author_id IN (...)` checks each row in `Books` against this list. For
very large lists, this `IN` operation can become inefficient.")
    print("2.  **JOIN:** The `JOIN` operation effectively combines the
`Books` and `Authors` tables based on matching `author_id`s. The
`WHERE` clause then filters these combined rows directly. Database
optimizers are highly efficient at handling JOINs, especially when
foreign key relationships are properly indexed (as
`idx_books_author_id` helps here).")
    print("While for this small dataset the time difference might be
negligible or even misleading due to overhead, the performance gain of
using JOINs over `IN` subqueries becomes significant with millions of
rows.")

demonstrate_query_optimization()
```

```
Tables 'Authors' and 'Books' created successfully.
Sample data inserted.
Index on Books.author_id created.
```

```
--- Querying for books by UK authors ---

Original Query Results (Books by UK authors):
(1, 'Pride and Prejudice', 1)
(2, 'Sense and Sensibility', 1)
(3, 'Great Expectations', 2)
(4, 'Oliver Twist', 2)
Original Query Time: 0.000060 seconds

Optimized Query Results (Books by UK authors):
(1, 'Pride and Prejudice', 1)
(2, 'Sense and Sensibility', 1)
(3, 'Great Expectations', 2)
(4, 'Oliver Twist', 2)
Optimized Query Time: 0.000046 seconds

Verification: Results are identical -  True

--- Explanation of Optimization ---
Both queries return the same correct results: all books written by
authors from the UK.
For larger datasets, the JOINed query (optimized version) is generally
faster than the subquery version. Here's why:
1.  **Subquery (IN clause):** The inner subquery `(SELECT author_id
FROM Authors WHERE country='UK')` is executed first to get a list of
author_ids. Then, the outer query `SELECT * FROM Books WHERE author_id
IN (...)` checks each row in `Books` against this list. For very large
lists, this `IN` operation can become inefficient.
2.  **JOIN:** The `JOIN` operation effectively combines the `Books`
and `Authors` tables based on matching `author_id`s. The `WHERE`
clause then filters these combined rows directly. Database optimizers
are highly efficient at handling JOINs, especially when foreign key
relationships are properly indexed (as `idx_books_author_id` helps
here).
While for this small dataset the time difference might be negligible
or even misleading due to overhead, the performance gain of using
JOINs over `IN` subqueries becomes significant with millions of rows.
```

Task 5: University Course Registration

Scenario: SR University needs a course registration system for students enrolling in different courses under faculty supervision.

Requirements:

1. **Schema Design:** Create normalized tables for `Students`, `Courses`, `Faculty`, and `Registrations`.
2. **Define Relationships:**
   - Each student can register for multiple courses (many-to-many relationship via `Registrations`).

    - Each course is taught by one faculty member (one-to-many relationship from `Faculty` to `Courses`).
3. **Sample Data:** Insert sample data that reflects these relationships.
4. **SQL Queries:** Write SQL queries to:
    - List all students enrolled in a specific course (e.g., 'Potions').
    - Find faculty members teaching more than 2 courses.
    - Retrieve courses with the highest number of registrations.

```
import sqlite3
from datetime import datetime

def setup_university_database():
    conn = sqlite3.connect(':memory:')  # Use in-memory database for
demonstration
    cursor = conn.cursor()

    print("Creating University Course Registration database
schema...")

    # 1. Create Tables
    cursor.execute('''
        CREATE TABLE Faculty (
            faculty_id INTEGER PRIMARY KEY AUTOINCREMENT,
            first_name TEXT NOT NULL,
            last_name TEXT NOT NULL,
            email TEXT UNIQUE NOT NULL
        );
    ''')

    cursor.execute('''
        CREATE TABLE Courses (
            course_id INTEGER PRIMARY KEY AUTOINCREMENT,
            course_name TEXT NOT NULL,
            faculty_id INTEGER NOT NULL,
            credits INTEGER NOT NULL CHECK (credits > 0),
            FOREIGN KEY (faculty_id) REFERENCES Faculty(faculty_id) ON
DELETE CASCADE,
            UNIQUE (course_name) -- Ensure course names are unique
        );
    ''')

    cursor.execute('''
        CREATE TABLE Students (
            student_id INTEGER PRIMARY KEY AUTOINCREMENT,
            first_name TEXT NOT NULL,
            last_name TEXT NOT NULL,
            email TEXT UNIQUE NOT NULL,
            enrollment_date DATE DEFAULT CURRENT_DATE
        );
    ''')
```

```
    cursor.execute('''
        CREATE TABLE Registrations (
            registration_id INTEGER PRIMARY KEY AUTOINCREMENT,
            student_id INTEGER NOT NULL,
            course_id INTEGER NOT NULL,
            registration_date DATETIME DEFAULT CURRENT_TIMESTAMP,
            FOREIGN KEY (student_id) REFERENCES Students(student_id)
ON DELETE CASCADE,
            FOREIGN KEY (course_id) REFERENCES Courses(course_id) ON
DELETE CASCADE,
            UNIQUE (student_id, course_id) -- A student can register
for a course only once
        );
    ''')

    print("Schema created successfully.")

    # 2. Insert Sample Data
    print("Inserting sample data...")

    cursor.execute("INSERT INTO Faculty (first_name, last_name, email)
VALUES ('Professor', 'Albus', 'albus.d@university.edu')")
    cursor.execute("INSERT INTO Faculty (first_name, last_name, email)
VALUES ('Dr.', 'Minerva', 'minerva.m@university.edu')")
    cursor.execute("INSERT INTO Faculty (first_name, last_name, email)
VALUES ('Prof.', 'Severus', 'severus.s@university.edu')")

    cursor.execute("INSERT INTO Courses (course_name, faculty_id,
credits) VALUES ('Potions', 3, 3)")
    cursor.execute("INSERT INTO Courses (course_name, faculty_id,
credits) VALUES ('Transfiguration', 2, 4)")
    cursor.execute("INSERT INTO Courses (course_name, faculty_id,
credits) VALUES ('Charms', 1, 3)")
    cursor.execute("INSERT INTO Courses (course_name, faculty_id,
credits) VALUES ('Defense Against the Dark Arts', 1, 4)")
    cursor.execute("INSERT INTO Courses (course_name, faculty_id,
credits) VALUES ('Herbology', 2, 3)")

    cursor.execute("INSERT INTO Students (first_name, last_name,
email) VALUES ('Harry', 'Potter', 'harry.p@student.edu')")
    cursor.execute("INSERT INTO Students (first_name, last_name,
email) VALUES ('Hermione', 'Granger', 'hermione.g@student.edu')")
    cursor.execute("INSERT INTO Students (first_name, last_name,
email) VALUES ('Ron', 'Weasley', 'ron.w@student.edu')")
    cursor.execute("INSERT INTO Students (first_name, last_name,
email) VALUES ('Draco', 'Malfoy', 'draco.m@student.edu')")

    cursor.execute("INSERT INTO Registrations (student_id, course_id)
VALUES (1, 1)") # Harry -> Potions
```

```
    cursor.execute("INSERT INTO Registrations (student_id, course_id)
VALUES (1, 2)") # Harry -> Transfiguration
    cursor.execute("INSERT INTO Registrations (student_id, course_id)
VALUES (1, 3)") # Harry -> Charms
    cursor.execute("INSERT INTO Registrations (student_id, course_id)
VALUES (2, 1)") # Hermione -> Potions
    cursor.execute("INSERT INTO Registrations (student_id, course_id)
VALUES (2, 2)") # Hermione -> Transfiguration
    cursor.execute("INSERT INTO Registrations (student_id, course_id)
VALUES (2, 3)") # Hermione -> Charms
    cursor.execute("INSERT INTO Registrations (student_id, course_id)
VALUES (2, 4)") # Hermione -> DADA
    cursor.execute("INSERT INTO Registrations (student_id, course_id)
VALUES (3, 1)") # Ron -> Potions
    cursor.execute("INSERT INTO Registrations (student_id, course_id)
VALUES (3, 3)") # Ron -> Charms
    cursor.execute("INSERT INTO Registrations (student_id, course_id)
VALUES (4, 1)") # Draco -> Potions
    cursor.execute("INSERT INTO Registrations (student_id, course_id)
VALUES (4, 4)") # Draco -> DADA

    conn.commit()
    print("Sample data inserted successfully.")

    # 3. Write SQL Queries

    # Query 1: List all students enrolled in a specific course (e.g.,
'Potions')
    print("\n--- Query 1: Students enrolled in 'Potions' ---")
    course_name_to_query = 'Potions'
    cursor.execute('''
        SELECT
            S.first_name, S.last_name, S.email
        FROM
            Students S
        JOIN
            Registrations R ON S.student_id = R.student_id
        JOIN
            Courses C ON R.course_id = C.course_id
        WHERE
            C.course_name = ?
        ORDER BY
            S.last_name, S.first_name;
    ''', (course_name_to_query,))
    students_in_course = cursor.fetchall()
    for student in students_in_course:
        print(student)

    # Query 2: Find faculty members teaching more than 2 courses
    print("\n--- Query 2: Faculty teaching more than 2 courses ---")
```

```
    cursor.execute('''
        SELECT
            F.first_name, F.last_name, F.email,
            COUNT(C.course_id) AS number_of_courses_taught
        FROM
            Faculty F
        JOIN
            Courses C ON F.faculty_id = C.faculty_id
        GROUP BY
            F.faculty_id, F.first_name, F.last_name, F.email
        HAVING
            COUNT(C.course_id) > 2
        ORDER BY
            number_of_courses_taught DESC;
    ''')
    faculty_teaching_many_courses = cursor.fetchall()
    for faculty in faculty_teaching_many_courses:
        print(faculty)

    # Query 3: Retrieve courses with the highest number of
registrations
    print("\n--- Query 3: Courses with the highest number of
registrations ---")
    cursor.execute('''
        SELECT
            C.course_name, C.credits,
            COUNT(R.registration_id) AS total_registrations
        FROM
            Courses C
        JOIN
            Registrations R ON C.course_id = R.course_id
        GROUP BY
            C.course_id, C.course_name, C.credits
        ORDER BY
            total_registrations DESC
        LIMIT 1;
    ''')
    top_registered_course = cursor.fetchone()
    print(top_registered_course)

    conn.close()
    print("\nDatabase connection closed.")

# Call the function to set up and query the database
setup_university_database()
```

```
Creating University Course Registration database schema...
Schema created successfully.
Inserting sample data...
Sample data inserted successfully.
```

```
--- Query 1: Students enrolled in 'Potions' ---
('Hermione', 'Granger', 'hermione.g@student.edu')
('Draco', 'Malfoy', 'draco.m@student.edu')
('Harry', 'Potter', 'harry.p@student.edu')
('Ron', 'Weasley', 'ron.w@student.edu')

--- Query 2: Faculty teaching more than 2 courses ---

--- Query 3: Courses with the highest number of registrations ---
('Potions', 3, 4)

Database connection closed.
```